

The Business Logic Layer (BLL) in our Email Automation System is responsible for processing emails, applying rules, and generating responses. It acts as an intermediary between the UI (dashboard & login pages) and backend/data components.

Core Functional Modules

1. Email Preprocessing Module

- Implemented in `preprocessor.py`
- Function: `clean_text()`
- Removes noise such as URLs, special characters, and converts text to lowercase.

This ensures consistent input before further processing.

2. Spam Detection Module

- Implemented in: `spam_detector.py`
- Function: `detect_spam()`
- Uses keyword-based filtering to identify spam emails.

If spam is detected, further processing is skipped.

3. Email Classification Module (ML Model)

- Implemented in: `ml_model.py`
- Function: `predict_category()`
- Uses TF-IDF + Logistic Regression to classify emails into:
- Complaint
- Request
- Inquiry
- Spam

Enables intelligent automation of responses.

4. Priority Assignment Module

- Implemented in: `priority_classifier.py`
- Function: `assign_priority()`
- Assigns priority levels:

Assign priority levels:

- Complaint → High
- Request → Medium
- Inquiry → Low

Helps in deciding the urgency of actions.

5. Response Generation Module

- Implemented in: response_generator.py
- Function: generate_response()
- Generates automated replies based on category and keywords.

Ensures contextual and meaningful responses.

6. Workflow Orchestration Module

- Implemented in: workflow.py
- Function: process_email()
- Integrates all modules:
- Preprocessing
- Spam detection
- Classification
- Priority assignment
- Response generation
- Database storage
- Email sending

This is the **core of BLL**.

Interaction with Presentation Layer (UI)

The UI is implemented using:

- login.html (user authentication)
- dashboard.html (main system interface)

Interaction Flow:

User enters email content in the dashboard

UI calls backend API:

POST /process_email

Backend calls:

process_email(content)

BLL processes email and returns:

- category
- priority
- response

UI displays the result dynamically

Example from dashboard:

- processEmail() function sends a request
- Result shown in <pre id = 'result'>

2.A)

Business Rules Implementation

Business rules define how the system behaves under different conditions.

Rules Implemented:

Spam Handling Rule:

- If spam is detected → mark it as "Spam."
- No reply sent

Implemented in detect_spam()

Category-Based Processing:

Email classified into Complaint / Request / Inquiry

- Using ML model (predict_category)

Priority Rule Complaint → High priority

- Request → Medium

Inquiry → Low priority

- inquiry → Low

Implemented in `assign_priority()`

Admin Escalation Rule:

If Complaint or High priority:

- Send to admin
- Do NOT auto-reply

Implemented in `workflow.py`

Auto-Reply Rule:

If not complaint:

Send an automated response to the user

👉 These rules ensure the system behaves intelligently and aligns with real-world email handling systems.

B) Validation Logic

Validation ensures that input data is correct and usable.

◆ **Implemented Validations:**

- **Input Validation (Frontend)** Input field ensures user enters content before processing
- 👉 Seen in dashboard UI
- **Text Cleaning Validation** Removes:
 - special characters
 - URLs
 - extra spaces
- 👉 Implemented in `clean_text()`
- **Spam Keyword Validation** Checks for words like:
 - "win", "lottery", "free", etc.
- 👉 Prevents malicious processing
- **Conditional Validation in Workflow** Ensures correct flow:
 - Spam → stop processing
 - Valid email → continue pipeline

👉 These validations ensure system reliability and prevent incorrect processing.

C) Data Transformation

Data transformation converts raw data into usable format for UI.

Transformations Implemented:

Raw → **Clean Text** Input email → cleaned version

- 👉 `clean_text()`

Text → **Feature Vector** TF-IDF converts text → numerical format

- 👉 Used in ML model

Prediction → **Structured Output:**

Output converted into JSON:

```
{
  cleaned_text,
  category,
  priority,
  response
}
```

Database Transformation

- Data stored in structured format:
- content
- cleaned_text
- category
- priority
- response